

Statistical Plan and Example Code (Python and R)

The Hackles Kit

Trust your hackles. Pause · Check · Tell.

FIELD COPY



PLATE I. The six strands, hand-inked, in the order the lessons take them.

Hackles · a free kit · quillsup.org

© 2026 Competence Collective. Lesson text and teaching materials: CC BY-NC 4.0 (free to print, share and adapt for non-commercial use with credit). The Hackles name, logos, characters and artwork are not covered by that license: print and share them unaltered for classroom and family use; any other use needs written permission. [VERIFY with counsel]



© 2026 Competence Collective. Lesson text and teaching materials CC BY-NC 4.0. The Hackles name, logos, characters and artwork are not covered by that license: print and share them unaltered for classroom and family use; any other use needs written permission. [VERIFY with counsel]

A simple, transparent plan for analyzing Hackles studies, with code you can run without installing anything. This is a requested toolkit deliverable (the analysis plan for your own study), not a report of results. The Python code uses only the standard library and was run and tested on synthetic data generated here. The R code mirrors it but has NOT been run, because R was not available when this toolkit was built. The synthetic data are invented and say nothing about whether Hackles works. Hackles has not been evaluated in a controlled trial.

No student accounts, no student data collected, ever. The primary analysis uses aggregate class tallies.

01 What the plan estimates

ITEM	DEFINITION
Primary outcome (students)	Tell-Readiness: percent of core-item responses that give the best answer. Core items: K-2 items 1, 4, 5; 3-5 items 1, 4, 7; 6-8 items 2, 5, 7; 9-12 items 1, 4, 10 (Assessment Pack Section 7)
Primary outcome (candidates)	Academy Part A mean (items 1 to 18, scale 1 to 5), with items 10 and 11 (response readiness) read separately
Estimand (Design B)	Difference in mean class-level change in Tell-Readiness between taught-first and delayed classes
Unit of analysis	The class (students are nested in classes, and the research team receives class tallies only)
What it is not	An estimate of behavior change, harm prevented, or effectiveness

02 Data layout

- **Class tallies** (one row per class): study code, condition, n pre and post, best-answer counts and answered counts per item and wave, core totals, matched-pair counts. Layout in Instruments and Codebook, R7.
Example: `code/data/synthetic_class_tallies_6-8.csv`.
- **Candidate survey** (one row per candidate per wave): a1 to a18, b19 to b21, k1 to k5. Example:
`code/data/synthetic_candidates_academy_survey.csv`.
- **Student-level file** (optional, only if the IRB-approved protocol allows de-identified student-level data):
`code/data/synthetic_student_level_6-8.csv`.

03 The steps

1. **Check and clean.** Confirm every class has n pre, n post and counts that do not exceed n . Do not drop a class. Flag any group under 10 students (suppress it).
2. **Describe.** For each condition, report students with pre and post surveys; item-level percent with the best answer; Tell-Readiness pre, post and change in percentage points. The Assessment Pack's tally sheet gives the same tables.
3. **Class-level change.** For each class, $\text{change} = \text{post Tell-Readiness} - \text{pre Tell-Readiness}$.
4. **Primary test.** Difference in mean class change, taught-first minus delayed. Inference by **exact randomization** (relabel classes in every possible way; with 4 versus 4 classes there are 70) because normal-theory tests are unreliable with so few classes. Report Welch's t-test and a **class bootstrap** interval as supporting results.
5. **Matched pairs.** In taught-first classes, count students whose core score went up, stayed or went down. Report counts. An exact sign (McNemar-style) test on up versus down is descriptive only, because it ignores class clustering.
6. **Missing data.** Skipped items drop out of that item's denominator (Assessment Pack rule). Students lost at post stay in pre percentages and out of matched analyses. Report both ns for every class.
7. **Clustering.** If student-level data exist, compute the ICC and design effect to plan the next study (`analyze_student_level.py`). With a few classes the ICC is very imprecise.
8. **Candidates.** Paired differences in Part A mean; mean change with a bootstrap interval; paired t-test and a sign-flip randomization test; Cohen's d_z ; item-level changes; knowledge snapshot change. Check the ceiling (share of 5s at pre).
9. **Multiplicity.** One primary outcome. Everything else secondary or exploratory.
10. **Sensitivity.** Weight classes by size; drop one class at a time; use a different core definition. Report whether conclusions change.
11. **Report** with the limits statement.

Stepped wedge. Use class-period means and a model with class and period effects (the code in `power_and_sample_size.py` shows the two-way fixed-effects estimator and randomization inference). A mixed model with a random class effect is the usual choice when student-level data are allowed (R: `lme4::lmer`; Python: `statsmodels`). These were not run here.

04 How to run (Python)

From the folder `kit/28_College/research/code`:

```
python generate_synthetic_data.py      # writes data/ (invented data, fixed seed)
python analyze_tallies.py              # primary analysis on class tallies
python analyze_candidates.py           # teacher candidates
python analyze_student_level.py        # optional ICC look
python power_and_sample_size.py        # sample size tables and simulation checks
python test_all.py                     # 31 checks; prints PASS or FAIL
python run_all.py                      # writes expected_output.txt
```

To analyze your own data, point the scripts at your CSVs: `python analyze_tallies.py path/to/your_tallies.csv`. Your file must use the same column names (see the Codebook, R7). Only the standard library is needed.

05 What the synthetic data show (an illustration)

THE GENERATOR INVENTS EIGHT grade 6-8 classes (four taught first, four delayed) with a known true gain built in, plus 36 invented candidates. These are the main results from `expected_output.txt`.

Class tallies (primary).

CONDITION	STUDENTS PRE	STUDENTS POST	TELL-READINESS PRE	POST	CHANGE
Taught first (4 classes)	86	77	48%	76%	+28
Delayed (4 classes)	91	87	51%	55%	+4

Class changes: taught first +24, +19, +32, +33; delayed +3, -1, +10, 0. The difference in mean class change is **+24.2 points**. Welch $t = 5.77$ (df 5.7), $p = 0.001$; exact randomization $p = 0.029$ (all 70 relabelings); class bootstrap 95 percent interval +16.0 to +31.4. Matched core-score pairs in taught-first classes: 28 up, 18 same, 1 down.

Why report the randomization p, not the Welch p. With only 8 classes the t-test's assumptions are fragile, and the exact p (0.029) is more honest than the Welch p (0.001). With 4 classes per arm the smallest possible exact p is 0.029, so even a perfect separation cannot do better.

Candidates. Part A mean rose from 2.57 to 3.26 (+0.69; bootstrap interval +0.53 to +0.83; $d_z = 1.52$; paired $t = 9.13$, df 35; $p < 0.0001$). The knowledge snapshot rose by more than a point of five. The invented gain is large by construction; real changes will usually be smaller and, without a comparison group, mean less.

ICC (student-level, optional). ICC of the pre core score 0.073; of the matched change score 0.083 (design effect 2.37 at 18 students per class). The estimate from eight classes is noisy, as expected.

What the example teaches. The method recovers the gain that was built in. It also shows how few classes limit precision. Real effects will not be this large, and real data will be messier.

06 Core code, Python

The class-level primary analysis, in short (from `analyze_tallies.py`):

```
import hackles_stats as hs

def tell_readiness(r, wave):          # r = one class row
    return 100 * r["core_%s_best" % wave] / r["core_%s_answered" % wave]
]

taught = [tell_readiness(r, "post") - tell_readiness(r, "pre") for r in
rows if r["condition"] == "taught_first"]
delayed = [tell_readiness(r, "post") - tell_readiness(r, "pre") for r in
rows if r["condition"] == "delayed"]

diff = hs.mean(taught) - hs.mean(delayed)
t, df, p_welch = hs.welch_t(taught, delayed)
p_rand = hs.perm_diff_in_means(taught, delayed)          # exact: every way
to relabel the classes
```

The candidate analysis, in short:

```
d = [avg(v["post"], A) - avg(v["pre"], A) for v in pairs.values()] #
A = a1..a18
m = hs.mean(d)
lo, hi = hs.bootstrap_ci_mean(d)
t, df, p = hs.paired_t(d)
dz = hs.cohens_dz(d)
p_rand = hs.sign_flip_p(d)
```

`hackles_stats.py` holds every helper: Student t p-values (incomplete beta function), exact McNemar, sign-flip and relabeling randomization tests, bootstrap, Wilson interval, ICC and sample-size formulas.

07 Core code, R

`code/analysis.R` (base R only). **Not run.** Key lines:

```

tallies$str_pre <- 100 * tallies$core_pre_best / tallies$core_pre_answered
tallies$str_post <- 100 * tallies$core_post_best / tallies$core_post_answered
tallies$change <- tallies$str_post - tallies$str_pre
a <- tallies$change[tallies$condition == "taught_first"]
b <- tallies$change[tallies$condition == "delayed"]
t.test(a, b) # Welch
idx <- combn(length(c(a, b)), length(a)) # exact randomization
perm <- apply(idx, 2, function(i) abs(mean(c(a, b)[i]) - mean(c(a, b)[-i]))))
mean(perm >= abs(mean(a) - mean(b)) - 1e-12)
binom.test(min(up, down), up + down, 0.5)$p.value # exact sign test
d <- rowMeans(post[, A]) - rowMeans(pre[, A]) # candidates
t.test(d) # paired t on differences

```

Compare with the Python output. Run the Python scripts first. In R, the percentages, the Welch t, the exact randomization p, the exact sign test, the paired t and dz should match `expected_output.txt` to the printed digits. The bootstrap intervals will differ in the last digit or two, because R and Python use different random number generators. If they differ by more than that, one of the scripts has a bug; please report it.

08 How the code was checked

`test_all.py` runs 31 checks and all passed in the build environment.

- **Statistics against known values.** Student t p-values at $t = 2.228$ and 2.0 (df 10), $t = 2.0$ (df 8), and $t = 12.706$ (df 1); t quantile $(.975, \text{df } 10) = 2.228$; paired t on 1 to 5 ($t = 4.243$, $p = .0132$); Welch t on two shifted samples; exact McNemar for 2 versus 10 equals $158/4096$; sign-flip p for 1 to 5 equals $2/32$; exact relabeling p for two sets of 3 equals $2/20$; the ICC for identical groups equals 1; the design effect; the Wilson interval.
- **Kit worked examples.** 26 of 72 to 51 of 72 is +35 points (Assessment Pack Section 14); 11 of 24 to 17 of 22 is +31 points (Academy Module 8).
- **Sample-size formulas against simulation.** 34 pairs for dz 0.5 gives simulated power about 0.82; 64 per group for d 0.5 gives about 0.81; 57 pairs for the McNemar example gives about 0.77 to 0.81 (the formula is slightly optimistic).
- **Synthetic data.** Reproducible (same seed, same files); the tally file has no student identifiers; the student file has codes and no names; the analysis recovers a gain in the plausible range for what was built in; outputs are deterministic; groups under 10 are suppressed.

What was not checked.

- The R code was never run.

- The pure-Python p-values were validated at a handful of textbook values, not against a full statistics package. For a publication, re-run the main analysis in R or a mainstream Python package and compare.
- The simulations make assumptions (normal errors, equal class sizes, no contamination, no carry-over). Real data will break some of them.
- The synthetic generator is a toy. It does not reproduce the item-level structure of real survey data.

09 Adapting to other bands and designs

- **Other bands.** Change the core item definitions (Assessment Pack Section 7) and the item list in the tally layout. The scripts read whatever columns you give them for core totals.
- **Stepped wedge.** Reuse `sw_theta()` and the randomization loop in `power_and_sample_size.py` on your class-period means.
- **Unequal class sizes.** Weight classes by matched n as a sensitivity analysis.
- **More classes.** Replace the exact randomization with a Monte Carlo version once the number of relabelings is large.

This document is educational guidance, not legal advice. Run it on invented data first. Never put real student data in a spreadsheet you cannot protect.